CL & NLP Assignment: LSTM Practice Questions

Sneha Chaudhary
B.Tech CSE (Data Science), Sem 7, UPES

I've used σ(z) = 1/(1 + e^(−z)) and tanh(z) = (e^z − e^(−z))/(e^z + e^(−z)), and rounded intermediate values to 4 decimal places like the sheet says.

LSTM equations I used:

```
f_t = σ(W_f·x_t + U_f·h_(t-1) + b_f)
i_t = σ(W_i·x_t + U_i·h_(t-1) + b_i)
C~_t = tanh(W_c·x_t + U_c·h_(t-1) + b_c)
C_t = f_t·C_(t-1) + i_t·C~_t
o_t = σ(W_o·x_t + U_o·h_(t-1) + b_o)
h_t = o_t·tanh(C_t)
```

Q1. Forward pass of a single LSTM cell

Given x_t = 0.5, h_(t-1) = 0.2, C_(t-1) = 0.4

Forget gate

```
z_f = 0.7(0.5) + 0.3(0.2) + 0.1 = 0.35 + 0.06 + 0.1 = 0.51
f_t = 1/(1 + e^(-0.51)) = 1/(1 + 0.6005) = 1/1.6005 = 0.6248
```

Input gate

```
z_i = 0.6(0.5) + 0.4(0.2) + 0 = 0.30 + 0.08 = 0.38
i_t = 1/(1 + e^(-0.38)) = 1/(1 + 0.6839) = 1/1.6839 = 0.5939
```

Candidate

```
z_c = 0.9(0.5) + 0.2(0.2) + 0 = 0.45 + 0.04 = 0.49
C~_t = tanh(0.49) = (e^0.98 - 1)/(e^0.98 + 1) = (2.6645 - 1)/(2.6645 + 1)
     = 1.6645/3.6645 = 0.4542
```

(I used tanh(z) = (e^2z − 1)/(e^2z + 1), which is the same formula, just quicker to calculate.)

New cell state

```
C_t = f_t·C_(t-1) + i_t·C~_t
    = 0.6248 × 0.4 + 0.5939 × 0.4542
    = 0.2499 + 0.2697
    = 0.5196
```

Output gate

```
z_o = 0.5(0.5) + 0.5(0.2) + 0.1 = 0.25 + 0.10 + 0.1 = 0.45
o_t = 1/(1 + e^(-0.45)) = 1/(1 + 0.6376) = 1/1.6376 = 0.6106
```

Hidden state

```
tanh(0.5196) = (e^1.0392 - 1)/(e^1.0392 + 1) = 1.8270/3.8270 = 0.4774
h_t = 0.6106 × 0.4774 = 0.2915
```

Final answers: f_t = 0.6248, i_t = 0.5939, C~_t = 0.4542, C_t = 0.5196, o_t = 0.6106, h_t = 0.2915

So the cell keeps around 62% of the old memory and adds about 59% of the new candidate, which is why the cell state goes up from 0.4 to 0.5196.

Q2. Number of parameters

Input size m = 10, hidden size n = 32. There are 4 gates and each one has input weights (n × m), recurrent weights (n × n) and a bias (n).

(a) One bias per gate:

```
4 × [n(m + n) + n] = 4 × [32(10 + 32) + 32]
                   = 4 × [1344 + 32]
                   = 4 × 1376
                   = 5504
```

So there are **5504** trainable parameters.

(b) PyTorch keeps two biases per gate (bias_ih and bias_hh), so each gate has an extra 32:

```
4 × [1344 + 2(32)] = 4 × 1408 = 5632
```

So **5632** parameters, which is 128 more than in (a).

Q3. Vanishing gradient through the cell state

Scaling factor ≈ f^k

(a) f = 0.9

- k = 10: 0.9^10 = **0.3487**
- k = 50: 0.9^50 = **0.0052** (about 5.15 × 10^-3)

(b) k = 50

- f = 0.5: 0.5^50 = **8.88 × 10^-16**, which is basically zero
- f = 0.99: 0.99^50 = **0.6050**

(c) For a dependency 50 steps back, the forget gate value matters a lot. With f = 0.5 the gradient basically disappears, and even with f = 0.9 only around 0.5% of it is left, so the model can't really learn that long-term link. But with f = 0.99 around 60% of the gradient survives, so it can still be learned.

In a normal RNN the gradient gets multiplied by the weight matrix and the tanh derivative at every step, and there's no way for the network to control this, so it vanishes quickly. In an LSTM the cell state is updated by adding ($C_t = f_t \cdot C_{(t-1)} + i_t \cdot \tilde{C}_t$), and the gradient along it mainly depends on the forget gate. Since the forget gate is learned, the network can keep it close to 1 when it needs to remember something, and the gradient can flow back over many steps.

Q4. Electricity load forecasting

Lookback = 168 hours, predict next 24 hours, 3 features.

(a) 3 × 365 × 24 = **26,280** hourly records

(b) Each window needs 168 + 24 = 192 hours in a row. With stride 1:

```
26,280 - 192 + 1 = 26,089 windows
```

(c) Taking batch first:

- input: **(64, 168, 3)** → 64 windows, 168 time steps, 3 features
- output: **(64, 24)** → 24 predicted load values for each window

(If the output keeps a feature dimension, it would be (64, 24, 1).)

(d) 70% of 26,280 = 18,396 records for training. Since this is time series data, I'm taking the first 70% in order and only counting windows that fit fully inside it:

```
18,396 - 192 + 1 = 18,205 training windows
18,205 / 64 = 284.45  →  285 mini-batches
```

That's 284 full batches plus one last batch of 29 windows (284 × 64 = 18,176 and 18,205 − 18,176 = 29).

(e) Hidden size 64, input 3, single bias:

```
4 × [64(3 + 64) + 64] = 4 × [4288 + 64] = 4 × 4352 = 17,408
```

So **17,408** parameters for the LSTM layer. (A dense layer from 64 to 24 outputs after it would add another 64 × 24 + 24 = 1,560, but the question only asks about the LSTM.)

Q5. Predictive maintenance

(a) Min-max normalisation:

```
(95 - 20)/(120 - 20) = 75/100 = 0.75
```

(b) 0.83 is more than the threshold of 0.5, so the model predicts that the pump **will fail** in the next cycle.

(c) From the question: 40 actual failures, 45 flagged, 30 of the flags correct.

- TP = 30
- FP = 45 − 30 = 15
- FN = 40 − 30 = 10
- TN = 1000 − 30 − 15 − 10 = 945

	Predicted fail	**Predicted OK**
Actually fails	30	10
Actually OK	15	945

```
Precision = 30/45 = 0.6667
Recall    = 30/40 = 0.75
F1        = 2(0.6667)(0.75)/(0.6667 + 0.75) = 1.0/1.4167 = 0.7059
Accuracy  = (30 + 945)/1000 = 0.975
```

(d) I think the team should focus on **recall**. Missing a real failure means the pump breaks down without warning, which causes downtime and repair costs and can even be a safety issue. A false alarm just means someone checks a pump that was fine, which is much cheaper. Right now the model still misses 10 out of 40 failures, so 1 in 4.

Accuracy is misleading here because failures are rare, only 40 out of 1000 (4%). A model that just says “no failure” every time would get 960/1000 = 96% accuracy without catching a single failure. So 97.5% sounds great but doesn’t tell us much. Recall, precision and F1 give a better idea, and lowering the threshold below 0.5 could be one way to improve recall (with more false alarms as the trade-off).